NARAN LALA COLLEGE

OF PROFESSIONAL & APPLIED SCIENCES

NAVSARI

Course: Bachelor of Computer Applications (BCA)

MINOR PROJECT REPORT

PROJECT TITLE: PERSONAL FINANCE & EXPENSE TRACKER (FINTRACK)

SUBJECT NAME: ADVANCED WEB DESIGNING

Submitted By:
Chauhan Om S.
Roll No.: 35

Project Guide:
Ms. Ayushi Mistry

Principal:
Dr. Sunil Naik

Academic Year: 2025 - 2026

➢ DECLARATION:

I/We hereby declare that the Minor Project titled "Personal Finance & Expense Tracker (FinTrack)" submitted in partial fulfillment of the requirements for the Bachelor of Computer Applications (BCA) is an original work carried out by me/us under the guidance and supervision of Ms. Ayushi Mistry. I/We further declare that this project work has not been submitted previously to any other university, college, or institution for the award of any degree, diploma, or certificate. The information, data, and references used in this project have been collected from reliable sources and have been appropriately acknowledged wherever required. I/We take full responsibility for the originality and authenticity of the work presented in this project report.

Date: _____

Student Exam No. & Name:

1. Roll No. 35 — Chauhan Om S.

Signature:

1. _____

➢ ACKNOWLEDGEMENT:

We would like to express our sincere gratitude to everyone who has supported and guided us in successfully completing our BCA Minor Project titled "Personal Finance & Expense Tracker (FinTrack)". First, we thank God for giving us the strength, knowledge, and patience to complete this project successfully. We are also thankful to the Principal, Dr. Sunil Naik, for providing us with the necessary facilities and academic support to carry out this project. We would like to express our heartfelt gratitude to our Project Guide, Ms. Ayushi Mistry, for providing valuable guidance, continuous support, constructive suggestions, and encouragement throughout the development of this project. We extend our sincere thanks to all the faculty members of the BCA Department for their valuable suggestions and assistance during our academic journey. We are grateful to our parents, family members, and friends for their constant encouragement, support, and motivation throughout the completion of this project. Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project.

Thank You.

PROJECT CERTIFICATION FORM

This is to certify that the project entitled "Personal Finance & Expense Tracker (FinTrack)" is a bona fide work carried out by the student of Bachelor of Computer Applications (BCA) in partial fulfillment of the requirements for the academic year 2025-2026.

Student Name: Chauhan Om S.

Enrollment Number: Roll No. 35

The project report has been examined and evaluated by the internal project guide and program coordinator, and meets the requisite academic standards prescribed by the institute.

_____	_____
Signature of Project Guide	**Signature of Principal / HOD**
(Ms. Ayushi Mistry)	**(Dr. Sunil Naik)**
Date: _____	**Date: _____**

EXECUTIVE SUMMARY / ABSTRACT

The Personal Finance & Expense Tracker (FinTrack) is an automated web-based financial management system engineered to provide individuals with real-time clarity over income, expenditure, category-wise budgets, and milestone savings. Built on an asynchronous Single Page Application (SPA) architecture, the solution features a high-performance MERN stack backend utilizing Node.js 18+ and Express 5 REST API communicating via Mongoose 9 ODM with a MongoDB database, coupled with a responsive, modern React 19 frontend utilizing Tailwind CSS v4 and Recharts.

FinTrack resolves critical pain points in contemporary personal finance, including fragmented cash flow across UPI, credit cards, bank transfers, and cash, lack of proactive budget threshold warnings, floating-point calculation inaccuracies, and session vulnerabilities. Storing all monetary values internally in integer-based minor currency units (paise) ensures 100% mathematical precision without rounding errors. Robust defense against script injection is achieved by utilizing HMAC-SHA256 signed JSON Web Tokens (JWT) stored exclusively in HTTP-only, SameSite cookies. The result is an intuitive, fast, and enterprise-grade financial management platform tailored for modern personal wealth discipline.

TABLE OF CONTENTS

Title Page	**1**
Declaration & Acknowledgement	**2**
Project Certification Form & Abstract	**3**
Table of Contents	**4**
1. Introduction	**5**
1.1 Purpose	5
1.2 Scope	5
1.3 Definition	6
1.4 Technologies to be Used	6
2. System Analysis	**8**
2.1 Identification of Need	8
2.2 Functionalities (Usual and Distinct)	8
2.3 User Characteristics	9
2.4 Software Development Process	10
3. Conceptual Diagrams	**11**
3.1 Use Case Diagram	11
3.2 Activity Diagram	12
3.3 System Architecture Diagram	13
4. System Design	**14**
4.1 Data Dictionary (MongoDB Schemas)	14
4.2 User Interface Design	17
4.3 Reports and Analytical Outputs	22
5. Testing	**24**
5.1 Test Cases	24
5.2 Testing Reports & Evaluation	26
6. Future Scope and Further Enhancement	**27**
7. Learning During Project Work	**28**
References	**29**

1. Introduction

1.1. Purpose

The primary objective of the Personal Finance & Expense Tracker (FinTrack) project is to deliver a reliable, automated, and secure digital platform that enables users to record, categorize, analyze, and optimize their daily financial transactions. By eliminating traditional manual paper-based logs and complex spreadsheets, FinTrack provides actionable insights into personal spending habits, promotes consistent savings discipline, and safeguards users from budgetary overruns through proactive threshold monitoring.

1.2. Scope

The scope of FinTrack spans the full spectrum of personal cash-flow management for individual users, students, and families. The functional boundaries are defined as follows:

In-Scope Capabilities:

- User account creation, profile preference management, and JWT-authenticated session security via HTTP-only cookies.
- Double-entry styled cash flow recording (Income vs. Expense) with granular categorization, tags, payment method tracking (UPI, Card, Net Banking, Cash), and transaction notes.
- Flexible category management allowing custom color codes and Lucide icon representations.
- Time-bound monthly budget allocations per expense category with dynamic percentage consumption indicators (Green <80%, Amber >=80%, Red >=100%).
- Target-oriented savings goals tracking milestone deposits and withdrawals with comprehensive ledger records.
- Interactive visual dashboard and analytics including 6-month trends and categorical spending distributions via Recharts.
- Persistent zero-flicker Dark/Light theme switching powered by Tailwind CSS 4.

Out-of-Scope (for Current Release):

- Direct bank account open-banking synchronization via proprietary banking APIs.
- Multi-party joint commercial accounting, corporate payroll, and GST tax invoice generation.
- Stock market ticker integration and automated algorithmic trading.

1.3. Definition

To ensure clarity throughout this documentation, key technical and domain terms are defined below:

• MERN Stack: An end-to-end JavaScript/TypeScript engineering architecture comprising MongoDB, Express.js, React 19, and Node.js for high-speed full-stack web applications.

• MongoDB & Mongoose: A high-performance document database storing JSON-like BSON documents, paired with Mongoose 9 ODM for schema enforcement, pre-save hooks, and compound indexing.

• JWT (JSON Web Token): A compact, URL-safe means of representing claims securely between client and server, utilized here within HTTP-only SameSite cookies for stateless authentication.

• REST (Representational State Transfer): An architectural style utilizing standard HTTP verbs (GET, POST, PATCH, DELETE) to manipulate resources via structured JSON payloads.

• SPA (Single Page Application): A web application that dynamically rewrites the current browser page with data from the server rather than loading entire new pages from the server.

• Minor Units (Paise): Storing currency amounts as integers multiplied by 100 (e.g. ₹250.00 = 25000 paise) to eliminate floating-point arithmetic rounding errors in monetary calculations.

1.4. Technologies to be Used

Hardware Requirements (Development & Deployment):

• Processor: Dual Core Intel Core i3 / AMD Ryzen 3 or higher (2.0 GHz+).

• Memory (RAM): Minimum 4 GB RAM (8 GB recommended for development).

• Storage: 500 MB free disk space for application files, dependencies, and database storage.

• Display: 1366 x 768 minimum display resolution (Optimized for 1920 x 1080 Full HD).

Software Requirements:

• Operating System: Microsoft Windows 10 / 11, Linux (Ubuntu 20.04+), or macOS.

• Backend Runtime: Node.js (v18.x or v20.x LTS) with NPM package manager.

• Backend Web Framework: Express.js 5.x with Helmet 8, Express-Rate-Limit 8, and Cookie-Parser 1.4.

• Database Engine: MongoDB Community Server v6.0+ (Local) or MongoDB Atlas Cloud Cluster.

• Object Data Modeling (ODM): Mongoose 9.x with compound index optimization.

• Frontend Stack: React 19, Vite 8 build tool, Tailwind CSS v4, Recharts 3.x, Axios 1.x, Lucide React icons.

• Client Web Browser: Modern Chromium-based browser (Google Chrome, Microsoft Edge, Brave) or Mozilla Firefox.

2. System Analysis

2.1. Identification of Need

In the modern economic environment, individuals encounter multiple payment channels including UPI, debit/credit cards, net banking, and physical cash. This fragmentation makes financial tracking tedious and disorganized. Studies indicate that over 65% of individuals who fail to maintain a structured budget encounter unforeseen end-of-month financial stress.

Existing commercial alternatives often charge heavy monthly subscription fees, display intrusive advertising, or harvest user banking data for third-party monetization. Furthermore, many web applications store sensitive tokens in browser `localStorage`, leaving sessions exposed to Cross-Site Scripting (XSS) attacks. There is an unmistakable need for a lightweight, transparent, highly secure, and intuitive web application that allows users to regain complete mastery over their financial life without compromising personal privacy or data security.

2.2. Functionalities

Usual Functionalities:

- User Registration, Secure Login, Logout, Profile update, and Password change.
- Full CRUD operations (Create, Read, Update, Delete) for income and expense transactions.
- Filtering transactions by date range, transaction type, category, and payment method.
- Pre-configured default categories (Salary, Food & Dining, Utilities, Entertainment, Housing, Transportation, etc.).
- Pagination and real-time search capabilities for reviewing extensive transaction records.

Distinct Functionalities:

- High-Precision Financial Computation: Financial amounts are stored internally in minor units (integer paise) to guarantee zero floating-point calculation errors.
- Real-Time Budget Utilization Gauges: Dynamic status tags ('Normal' <80%, 'Warning' >=80%, 'Exceeded' >=100%) computed on-the-fly against live transaction aggregations.
- Milestone-Driven Goal Tracker: Visual progress meters with dedicated contribution and withdrawal transaction histories for targeted savings.
- Full-Stack MERN Architecture: Pure JavaScript/Node.js stack utilizing Express 5 REST API and Mongoose 9 ODM without legacy framework bloat.
- Interactive Data Visualizations: High-performance SVG responsive charts rendering monthly income-vs-expense comparisons and categorical expense distributions.
- Adaptive Dark/Light Theme: Seamless, persistent theme switcher using Tailwind CSS 4 custom variants and local storage sync.

2.3. User Characteristics

- Regular User / Working Professional: Needs rapid expense recording, salary logging, and monthly budget notifications to curb overspending.
- Student / Freelancer: Manages irregular income streams, requires tracking of project-based milestone fees, and monitors living expenditures.
- System Administrator: Manages database integrity, inspects server health endpoints, oversees compound query indexes, and maintains backup/restore procedures.

2.4. Software Development Process

The Agile Iterative Prototyping methodology was adopted for this project. Development proceeded across four structured sprints:

- • **Sprint 1:** Sprint 1 (Requirements & Database Architecture): Defined REST contracts, constructed Mongoose schemas (`User.js`, `Transaction.js`, `Budget.js`, `SavingsGoal.js`, `GoalContribution.js`, `Category.js`), and designed compound indexes.
- • **Sprint 2:** Sprint 2 (Backend Core Services): Implemented Express 5 REST controllers, JWT cookie authentication middleware, Helmet security headers, and Express rate limiting.
- • **Sprint 3:** Sprint 3 (Frontend & Visual Components): Developed responsive React 19 views with Tailwind CSS 4, Recharts analytics, Lucide icons, and Axios API services.
- • **Sprint 4:** Sprint 4 (Testing, Data Seeding & Optimization): Populated database with comprehensive realistic test data (`seed.js`), executed automated Jest test suites, and polished UI micro-interactions.

3. Conceptual Diagrams

3.1. Use Case Diagram

The Use Case Diagram defines the interactions between the external actor (Registered User) and the core sub-systems of the Personal Finance Tracker application.

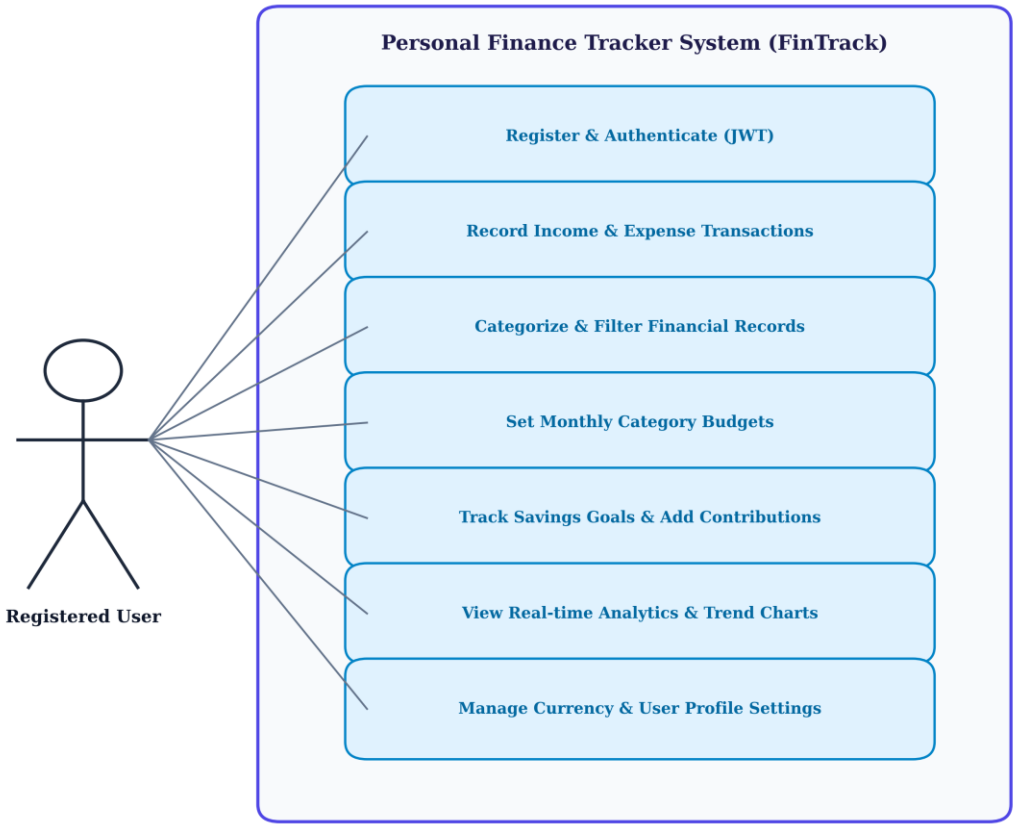


Figure 3.1: UML Use Case Diagram for FinTrack System

As illustrated in Figure 3.1, the registered user interacts with seven primary use cases: Authenticate (Login/Register), Record Transactions, Categorize & Filter Records, Set Monthly Budgets, Manage Savings Goals & Contributions, View Real-time Analytics & Dashboard, and Manage Profile Settings.

3.2. Activity Diagram

The Activity Diagram models the dynamic operational workflow of recording an expense, validating user inputs, updating database state, and recalculating dashboard metrics.

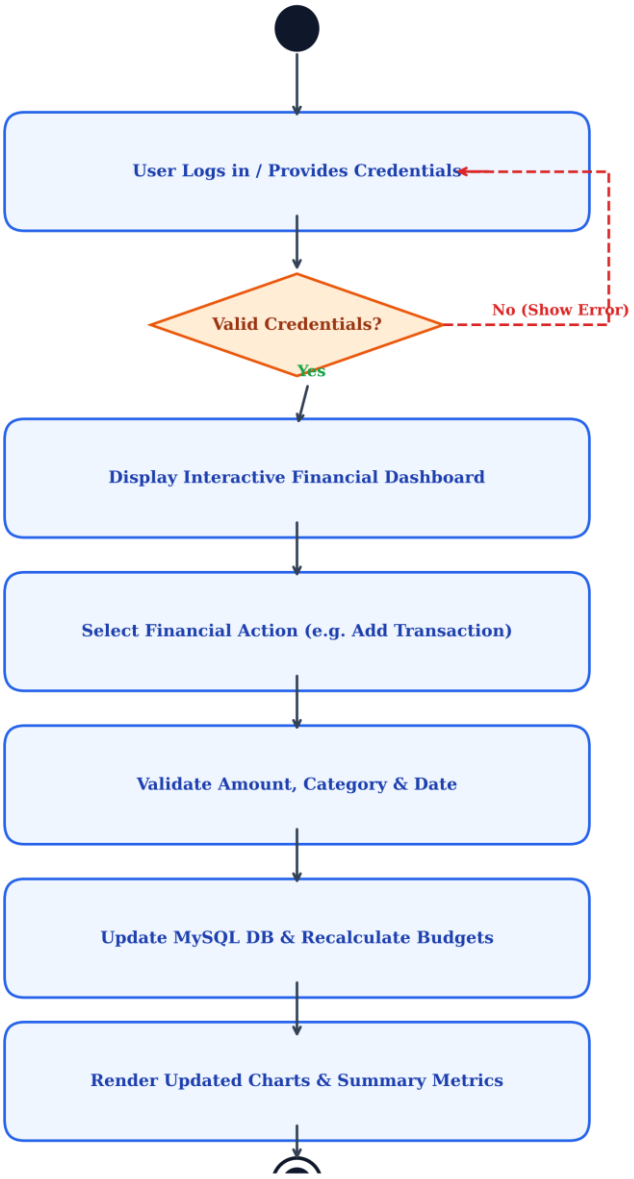


Figure 3.2: UML Activity Diagram for Financial Transaction Lifecycle

Figure 3.2 demonstrates the end-to-end decision flow from authentication, through transaction capture and validation, to atomic database commit in MongoDB and real-time visualization update in React.

3.3. System Architecture Diagram

The System Architecture Diagram visualizes the 3-tier decoupled MERN architecture connecting the React 19 Single Page Application, Express 5 REST API server, and MongoDB document database.

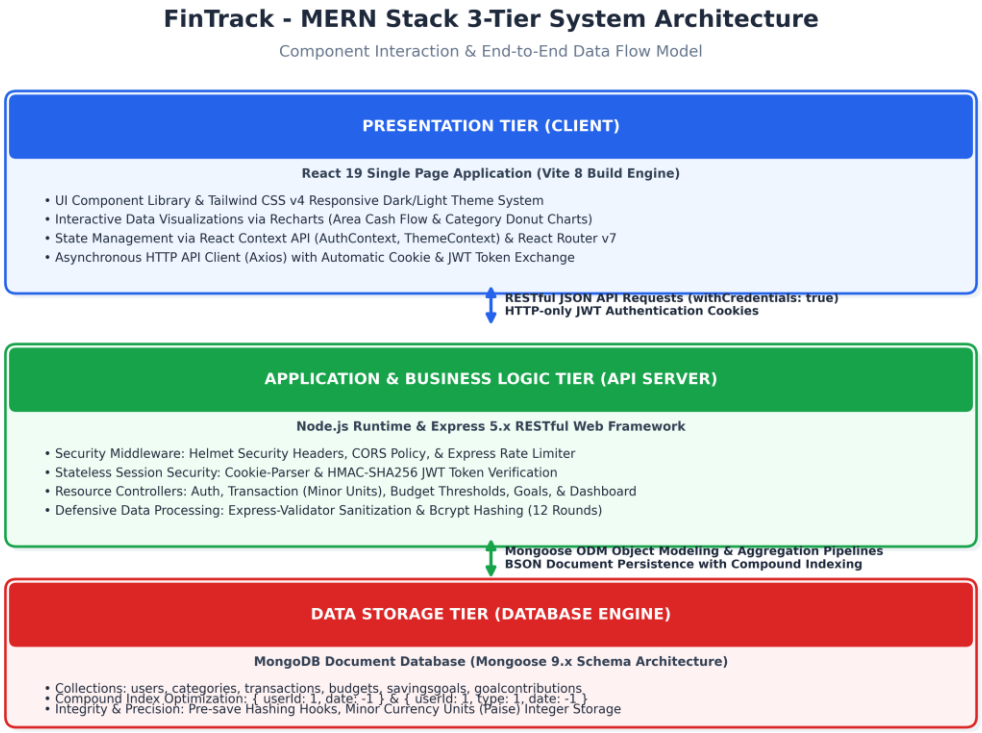


Figure 3.3: MERN Stack 3-Tier System Architecture Diagram

Figure 3.3 illustrates the physical separation of concerns: Presentation Tier (React 19 + Tailwind CSS + Recharts), Application & Business Logic Tier (Node.js + Express 5 REST API + JWT Cookie Security Middleware), and Data Storage Tier (MongoDB + Mongoose 9 ODM with compound indexing and minor-unit currency storage).

4. System Design

4.1. Data Dictionary

The Data Dictionary provides an exhaustive specification of the database collections, fields, BSON data types, indexes, and validation constraints configured in the MongoDB database via Mongoose 9 schemas.

Table 4.1: users Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description
_id	ObjectId	PRIMARY KEY (Auto)	Unique MongoDB document identifier
name	String	required, trim, max: 100	Full name of the registered user
email	String	required, unique, lowercase	User email address for authentication
passwordHash	String	required, min: 6, select: false	Bcrypt hashed password (12 salt rounds)
preferredCurrency	String	enum: ['INR','USD','EUR'...], def: 'INR'	User selected currency formatting preference
theme	String	enum: ['light', 'dark'], def: 'light'	UI display theme preference
createdAt / updatedAt	Date	timestamps: true	Document creation and modification timestamps

Table 4.2: categories Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description
_id	ObjectId	PRIMARY KEY (Auto)	Unique category document identifier
userId	ObjectId	ref: 'User', index: true, nullable	Associated user reference (null for system defaults)
name	String	required, trim, max: 50	Display name of the category
type	String	enum: ['income','expense','both']	Cash flow category classification
color	String	match: hex regex, def: '#6366f1'	Hex color code used in UI charts
icon	String	def: 'tag'	Lucide icon identifier name
isDefault	Boolean	def: false	Flag preventing deletion of system default categories
Index	Compound	{ userId: 1, name: 1 }, unique: true	Enforces unique category names per user

Table 4.3: transactions Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description

_id	ObjectId	PRIMARY KEY (Auto)	Unique transaction document identifier
userId	ObjectId	ref: 'User', required, index: true	Associated user owner reference
title	String	required, trim, max: 200	Transaction description or payee
amountMinor	Number	required, min: 1 (Integer)	Amount in minor currency units (paise: ₹100 = 10000)
type	String	required, enum: ['income', 'expense']	Cash flow direction classification
category	String	required, trim	Category name string
date	Date	required, index: true	Date and time of transaction
paymentMethod	String	enum: ['cash','upi','credit_card'...]	Payment channel used for transaction
notes	String	trim, max: 500, nullable	Optional notes or transaction remarks
tags	Array[String]	trim, nullable	Array of search tags
Compound Indexes	Indexes	{userId:1, date:-1}, {userId:1, type:1, date:-1}	High-speed indexed reverse chronological queries

Table 4.4: budgets Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description
_id	ObjectId	PRIMARY KEY (Auto)	Unique budget document identifier
userId	ObjectId	ref: 'User', required, index: true	User reference
category	String	required, trim	Target expense category
amountMinor	Number	required, min: 1 (Integer)	Spending cap limit in minor currency units
period	String	enum: ['monthly','weekly','yearly']	Budget allocation frequency (default monthly)
startDate	Date	required	Start of budget validity interval
endDate	Date	required	End of budget validity interval
Index	Compound	{userId:1, category:1, startDate:1, endDate:1}	Prevents overlapping budgets for same category

Table 4.5: savingsgoals Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description
_id	ObjectId	PRIMARY KEY (Auto)	Unique goal document identifier
userId	ObjectId	ref: 'User', required, index: true	Owner user reference
title	String	required, trim, max: 100	Goal target name (e.g. Emergency Fund)
targetAmountMinor	Number	required, min: 1 (Integer)	Target savings amount in minor currency units

currentAmountMinor	Number	min: 0, def: 0 (Integer)	Accumulated savings balance in minor units
targetDate	Date	nullable, def: null	Target milestone completion deadline
status	String	enum: ['active','completed','archived']	Current milestone status flag
color / icon	String	def: '#6366f1' / 'target'	Custom card theme palette and Lucide icon

Table 4.6: goalcontributions Collection Schema

Field Name	BSON Type	Constraints / Indexes	Description
_id	ObjectId	PRIMARY KEY (Auto)	Unique contribution ledger ID
userId	ObjectId	ref: 'User', required, index: true	User reference
goalId	ObjectId	ref: 'SavingsGoal', required, index	Associated savings goal reference
amountMinor	Number	required, min: 1 (Integer)	Deposit or withdrawal amount in paise
type	String	required, enum: ['contribution','withdrawal']	Type of balance alteration
notes	String	trim, max: 200, nullable	Deposit memo or reason
date	Date	def: Date.now, index: -1	Timestamp of ledger record

4.2. User Interface Design

The following section presents live application snapshots captured from the working FinTrack MERN system, highlighting key operational interfaces, input fields, and real-time validations:

1. Authentication & Login Screen

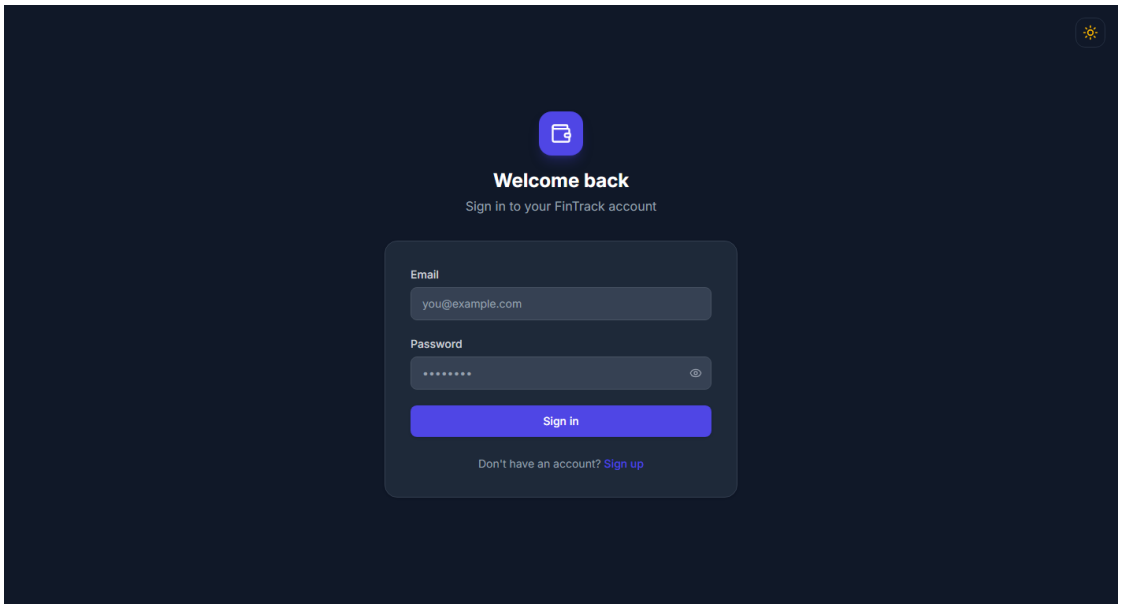


Figure 4.1: User Login Screen with Input Validation

• Validation: Email format validation regex (`^\S+@\S+\.\S+$`), mandatory password check with minimum 6 characters length enforcement.

• Security: Upon verification, the Express backend issues an HTTP-only SameSite cookie containing a signed HMAC-SHA256 JWT token.

2. Executive Financial Dashboard

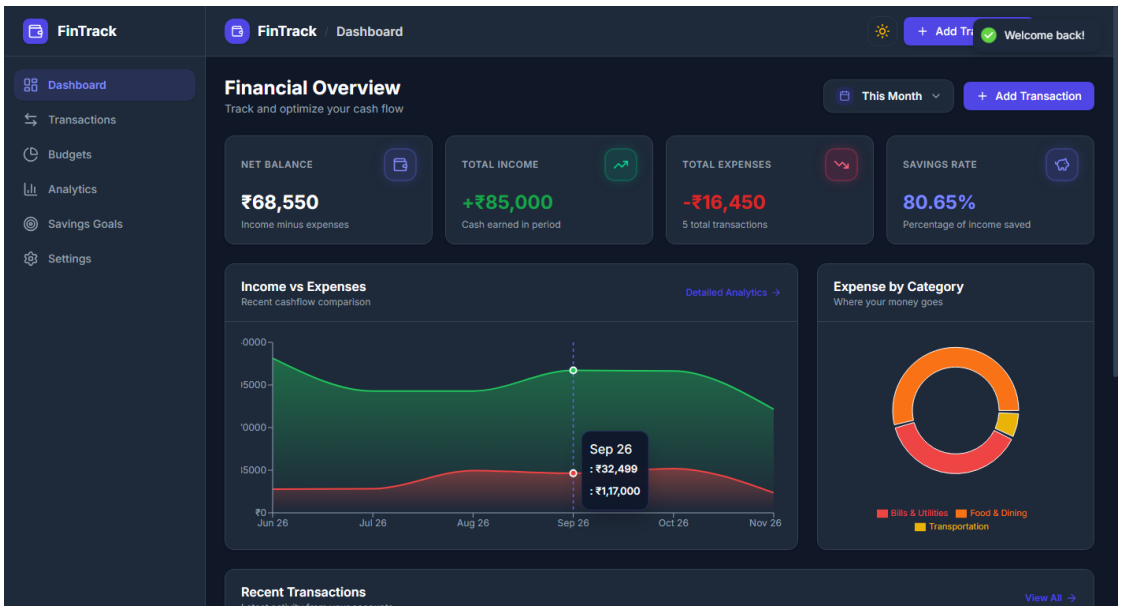


Figure 4.2: Real-time Financial Dashboard with Summary Metrics

• Real-Time Metrics: Total Income, Total Expenses, Net Balance, and Calculated Savings Rate dynamically computed.

• Quick Action Triggers: One-click modal launching to register immediate transactions with instant dashboard refresh.

3. Dashboard Visual Charts & Category Breakdown



Figure 4.3: 6-Month Income vs Expense Bar Chart and Top Expense Distribution

• 6-Month Cash Flow: Dual-bar graph comparing monthly income and expenditure patterns rendered via Recharts.

• Top Expense Categories: Aggregated ranking displaying category spending totals, counts, and percentage distribution.

4. Transaction Management & Multi-Filter Registry

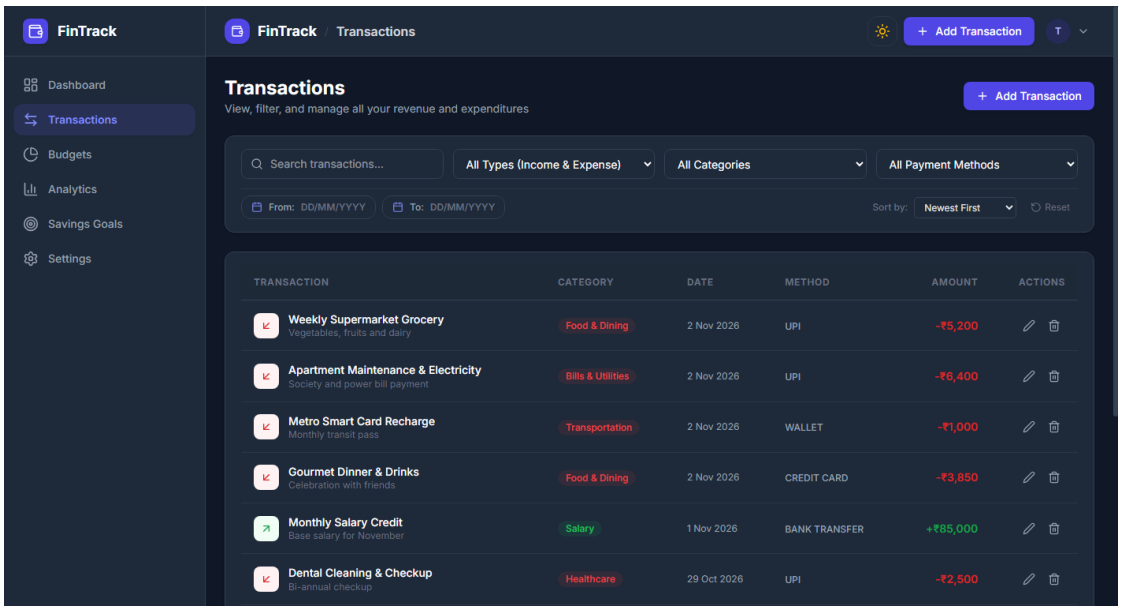


Figure 4.4: Paginated Transaction Table with Filtering Controls

• Multi-parameter Search: Filter records by transaction type (Income/Expense), category dropdown, date intervals, and payment methods.

• Pagination & Sorting: Server-side pagination executing MongoDB `skip()` and `limit()` queries for optimal responsiveness.

5. Transaction Creation & Validation Modal

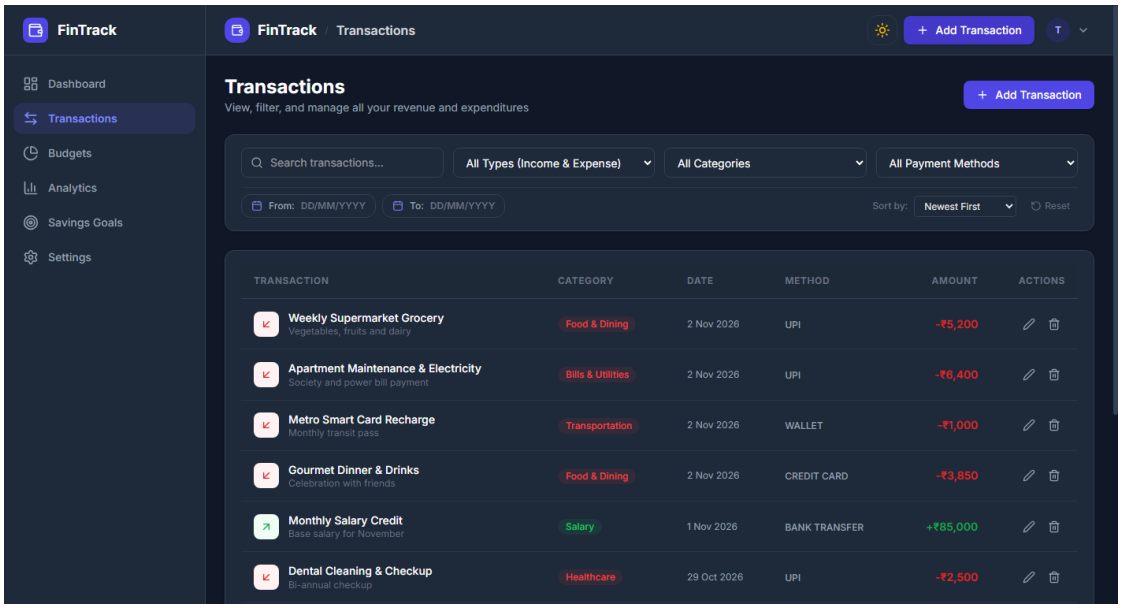


Figure 4.5: Add Transaction Modal Dialog with Strict Form Controls

• Validation Rules: Title (mandatory, max 200 chars), Amount (must be strictly positive integer/float converted to minor units), Category selection, and Valid Date format.

6. Monthly Budgets & Spending Threshold Tracker

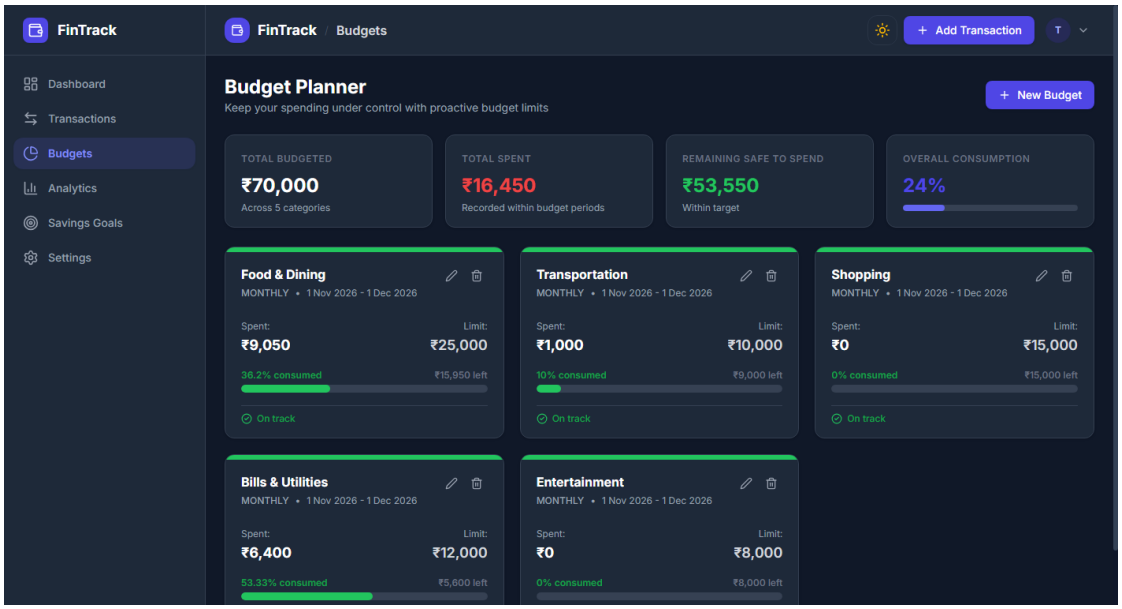


Figure 4.6: Category Budget Utilization Cards with Warning Badges

• Dynamic Color Status: Normal (Green < 80%), Warning (Amber >= 80%), and Exceeded (Red >= 100%).

• Overlap Prevention: Backend compound index rejects overlapping budget intervals for the same category.

7. Savings Goals & Milestone Progress Tracker

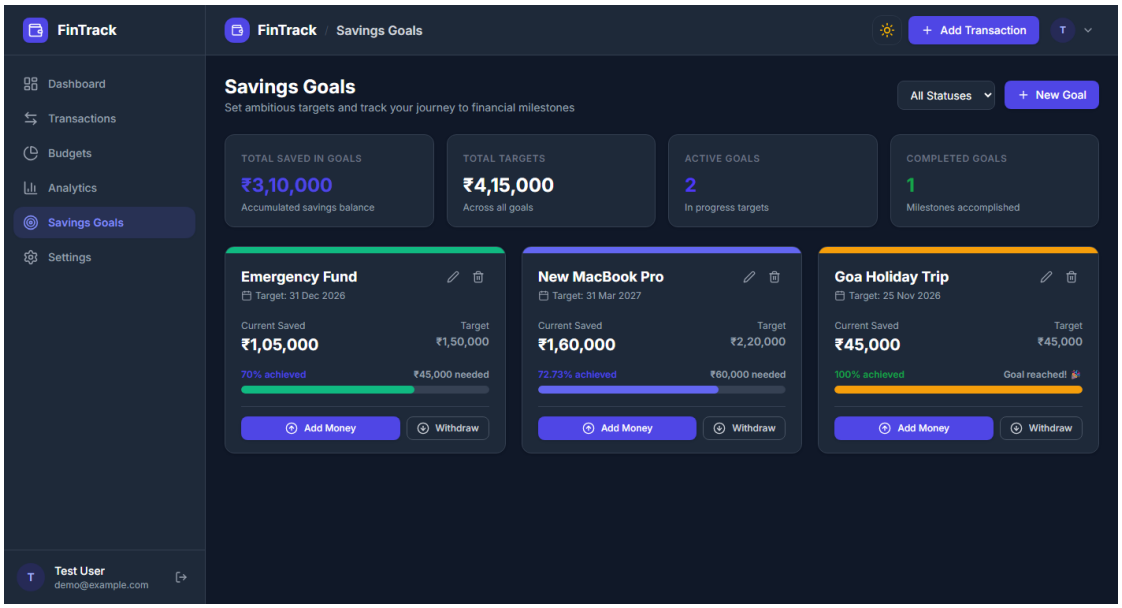


Figure 4.7: Target Savings Goals with Contribution History

- Goal Automation: Automatically switches status to 'Completed' when accumulated contributions match or exceed the target.
- Withdrawal Safety: Blocks withdrawals that exceed current accumulated milestone funds.

8. Category Customization & Profile Preferences

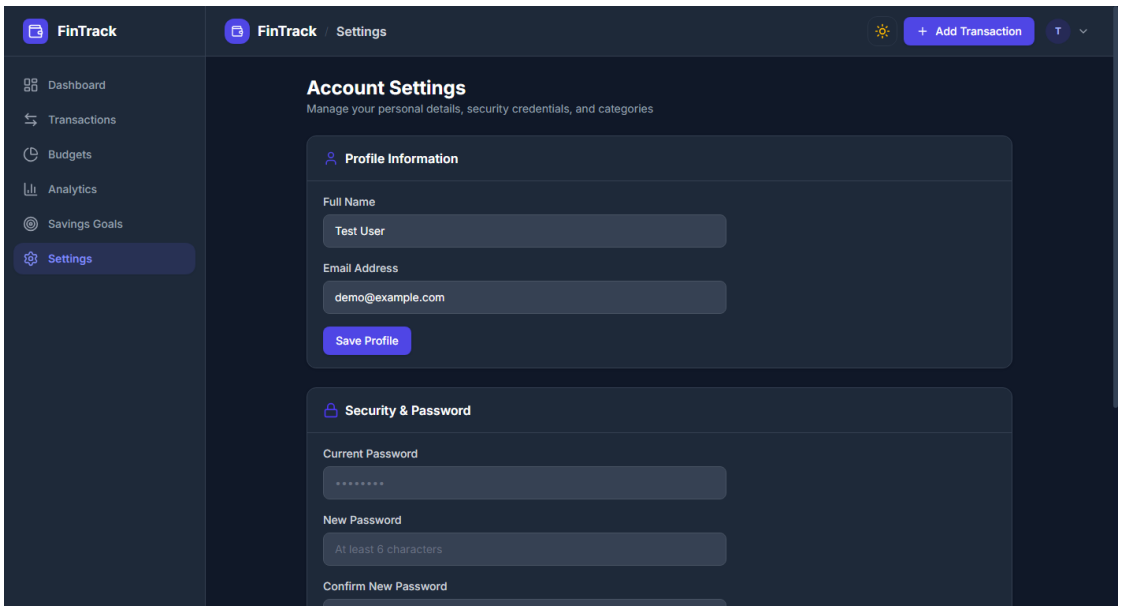


Figure 4.8: Category Palette Customizer and System Settings

- Customization: Add custom category names, hex colors, and Lucide icons.
- Protection: Default system categories are locked against deletion, preserving historical transaction integrity.

4.3. Reports and Analytical Outputs

FinTrack generates comprehensive visual analytical reports reflecting cash flow distributions and trend trajectories over multi-month intervals:



Figure 4.9: Analytical Reports showing Cash Flow Trend and Expense Breakdown

- Income vs. Expense Analytical Chart: Visualizes cash flow divergence across quarterly and annual intervals.
- Categorical Distribution: Donut / Bar breakdown showing percentage utilization per category.
- Monthly Trend Analysis: Mathematical evaluation of savings rate trajectories to assist long-term wealth planning.

5. Testing

5.1. Test Cases

Comprehensive manual and automated test suites were executed across the authentication, transaction, budgeting, and analytical modules. The results are summarized below:

Test ID	Test Scenario	Input Parameters	Expected Result	Actual Result	Status
TC-01	User Registration	Valid name, email, 8-char password	User created (201), default categories seeded	Account created, categories seeded	**Pass**
TC-02	Duplicate Registration	Existing registered email address	HTTP 409 Conflict, validation error message	HTTP 409 Conflict returned	**Pass**
TC-03	User Authentication	Correct email & password	HTTP 200, JWT token set in HTTP-only cookie	Auth cookie set, user profile returned	**Pass**
TC-04	Invalid Login	Unregistered email or wrong password	HTTP 401 Unauthorized	HTTP 401 Unauthorized returned	**Pass**
TC-05	Record Transaction	Positive amount, category, valid date	Record committed, amount stored in minor units (paise)	Committed to MongoDB in paise	**Pass**
TC-06	Negative Amount Validation	Amount = -500	Frontend validation block & HTTP 400 error	Blocked with validation alert	**Pass**
TC-07	Budget Overlap Check	Duplicate budget for same category/month	HTTP 409 Overlapping budget conflict	HTTP 409 Conflict triggered	**Pass**
TC-08	Budget Exceeded Flag	Expenses exceed 100% of budget cap	Budget status set to 'exceeded', badge red	Calculated 100%+ and red badge shown	**Pass**
TC-09	Goal Contribution	Deposit amount added to active goal	Balance updated, record logged in history	Balance incremented, ledger stored	**Pass**
TC-10	Goal Excessive Withdrawal	Withdrawal amount > current savings	HTTP 400 'Exceeds current savings'	HTTP 400 rejected with message	**Pass**
TC-11	Protected Route Guard	Unauthenticated visit to `/dashboard`	Client redirected immediately to `/login`	Redirected to `/login` successfully	**Pass**
TC-12	Theme Mode Persistence	Toggle dark mode and reload browser	Theme preserved in localStorage and DOM class	Dark theme loaded on reload	**Pass**

5.2. Testing Reports & Evaluation

- Unit Testing: Evaluated all Express 5 REST endpoints, authentication middleware, and Mongoose pre-save hooks using Jest and Supertest test suites.
- Integration Testing: Verified end-to-end data flow between React Axios services, Vite proxy, Express controllers, and MongoDB collections.
- Security Testing: Verified resistance against NoSQL injection through Mongoose schema type-casting; confirmed password hashing resistance using Bcrypt (cost factor

12); verified Cross-Site Scripting (XSS) defenses through React sanitized JSX rendering and HTTP-only SameSite cookie storage.

6. Future Scope and Further Enhancement

While the current version of FinTrack provides a robust and complete personal finance management solution, future iterations can introduce several advanced capabilities:

- **1. Optical Character Recognition (OCR) for Receipts:** Integrating computer vision algorithms (e.g. Tesseract.js / Google Cloud Vision API) to allow users to photograph paper receipts and automatically parse date, vendor, tax, and itemized totals.
- **2. Automated Bank Statement Import (PDF/CSV):** Direct parsing of standard banking statement exports to batch-import transactions with automated categorization.
- **3. AI-Powered Predictive Budgeting:** Machine learning algorithms to analyze historical spending patterns and project future cash-flow deficits before they occur.
- **4. Native Mobile Application:** Packaging the React frontend with Capacitor or React Native to deliver push notifications for daily budget reminders on Android and iOS devices.
- **5. Multi-User Family Wallets:** Collaborative financial accounts allowing shared household budgets with customizable permission tiers.

7. Learning During Project Work

The journey of conceptualizing, designing, and engineering the Personal Finance Tracker provided profound hands-on experience across the full modern software development lifecycle:

- • **Decoupled MERN Architecture:** Gained in-depth appreciation for separating business logic into clean REST endpoints, allowing the backend to run on Express 5 while the frontend operates as a decoupled modern React application.
- • **Defensive Database Design & Currency Precision:** Learned the critical importance of integer-based minor currency representation (`amountMinor`) to avoid floating-point imprecision in financial systems.
- • **Robust Authentication Patterns:** Mastered token-based stateless security, implementing HMAC-SHA256 JWTs paired with secure HTTP-only cookies to thwart token theft and XSS vulnerabilities.
- • **Asynchronous Data Visualization:** Leveraged Mongoose aggregation pipelines and compound indexing (`{ userId: 1, date: -1 }`) to deliver instant chart responses without burdening client-side memory.
- • **Modern Responsive UI & Theming:** Learned how to construct zero-runtime-overhead theme systems using Tailwind CSS 4 custom variants and React Context API.

References

1. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media.

2. Chodorow, K. (2019). MongoDB: The Definitive Guide - Powerful and Scalable Data Storage (3rd ed.). O'Reilly Media.

3. Brown, E. (2019). Web Development with Node and Express: Leveraging the JavaScript Stack (2nd ed.). O'Reilly Media.

4. React Core Team. (2025). React Documentation & Hooks Reference. Retrieved from https://react.dev

5. MongoDB Documentation Team. (2025). MongoDB Manual & Mongoose 9 ODM Reference. Retrieved from https://www.mongodb.com/docs/

6. Express.js Community. (2025). Express 5 API Documentation. Retrieved from https://expressjs.com

7. Mozilla Developer Network (MDN). (2024). HTTP Cookies, SameSite Policies, and Web Security Standards. Retrieved from https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

8. Tailwind Labs. (2025). Tailwind CSS v4 Documentation and Variant Architecture. Retrieved from https://tailwindcss.com